

Lab 07: Shaders

CP479 Computer Graphics

September 18, 2026

Last week the shaders were given and you wrote the **data**: five buffers, laid out to the byte. This week it is the other way round. Every line of C++ in the skeleton is finished. The only files you edit are six shaders, and the first of them fills the screen with **no vertex buffer at all**: the vertex stage reads three positions out of `gl_VertexIndex`, so last week's five buffers collapse to none and all of the work moves into the fragment shader. Everything after that is a function of a pixel and a clock.

AI coding assistants are permitted and expected. You must be able to explain everything you ship. You must also keep an AI prompt log: for every prompt, record the idea behind it and how you verified the answer (see Section 9). A circle, a value noise and a smooth minimum are among the most copy-pasteable twenty lines in graphics, so be clear about where the work actually is: six shader files, and five write-ups about numbers that do not exist anywhere until *you* have run the measurement on *your* machine. An assistant can hand you the first; nobody can hand you the second. In the last ten minutes of the session you will be asked to change one constant in your own Part V shader, live, and say what will happen before you press save.

1 Setup

Nothing new to install. The Vulkan SDK, CMake and GLFW you set up for Lab 06 are the whole toolchain again, and the one extra thing this lab leans on, `glslc`, came with the SDK. If Lab 06 built, this builds.

1.1 Get the skeleton and build it

<https://github.com/2110479-Computer-Graphics/Lab07-Shaders>

Press *Use this template* to make your own copy, then clone that copy. Your work lives in your repository, so commit as you go. Then run *one* of these.

```
make info                # macOS, Linux
make.bat info            # Windows
cmake -S . -B build && cmake --build build --target info # anywhere
```

The parts are `info` `circle` `terrain` `sdf` `heightmap` `toy`, one program each, so a part that fails to build cannot stop the others. The build compiles every `.vert` and `.frag` in `shaders/` with `glslc`. The `.glsl` files are includes: they are never compiled on their own, but every compiled shader depends on them, so touching `noise.glsl` rebuilds everything that includes it.

1.2 What make info tells you

```
$ make info
device      Apple M2 Pro
api         1.3.334
subgroup    32 lanes
timestamps  1 ns per tick
vertex inputs 31 attributes max, and Part I uses none of them
```

```

glslc          /usr/local/bin/glslc
shaders        .../lab07-skeleton/shaders
circle.frag     TASK 1 incomplete
noise.glsl     TASK 2 incomplete
...

```

The last block is one line per file you edit. **incomplete** means the file still contains a **TODO**(TASK marker; **done** means you deleted it. It is a grep, not a grade: deleting the comment without writing the code still says **done**, and the panels will still say the picture is wrong. *timestamps* must be a number, or Parts II to V cannot measure anything. Keep this screenshot for your report.

1.3 Hot reload, keys and flags

Every part prints its panel, then opens a window. While that window is open the host checks the timestamps of every file in **shaders/** four times a second. Save a shader and it recompiles and rebuilds the pipeline without closing anything:

```

reloaded      sdf.frag
compile error sdf.frag:41: error: 'k' : undeclared identifier

```

A failed compile costs you a message and nothing else: the old pipeline keeps drawing, so you fix the typo, save again, and the picture updates. Edit with the window open. It is the fastest loop in this course so far.

Every part answers the same keys while the window is open, and the same flags on the command line. **make** passes flags through with **ARGS**.

Key or flag	What it does
0 1 2 3	the four debug views, the mode uniform (§7.1.5)
Up Down	octaves, 1 to 8
S / Esc	saves report/<part>.png / quits
[]	Part IV only: halve or double the grid N , which rebuilds the mesh
left mouse	Part III: drag the ball. Part V: iMouse
--time T	freeze the clock at T seconds, for the panel and the window
--headless	no window, exit after the panel
--save p.png	write the frame the panel measured
--octaves n	Parts II, IV and V
--grid n	Part IV's N
--mode m	which view the panel measures

```

make circle ARGS="--time 0"
make circle ARGS="--mode 1 --headless --save uv.png"
make heightmap ARGS="--grid 32 --mode 2"
make.bat circle --time 0          # Windows takes them directly

```

The panel always measures at **time** = 1.0 unless you say otherwise, because the reference images were rendered at $t = 1$. That is why **make circle** can compare your picture against a file and tell you it matches.

1.4 If it does not work

What you see	What to do
CMake: no GLSL compiler found	<code>glslc</code> is in the SDK's <code>bin</code> . Open a new terminal, or on macOS run the SDK's <code>setup-env.sh</code> .
'#include' : must be enabled, cannot open include file	The file lost its <code>#extension</code> line, the one naming <code>GL_GOOGLE_include_directive</code> , or it used quotes. Write <code>#include <noise.glsl></code> : angle brackets search the include path the build sets up, quotes look next to the file instead.
timestamps UNSUPPORTED	The driver cannot time a pass, so every <code>gpu time</code> line is meaningless. On macOS this means the SDK is old: reinstall it. Tell us before you submit timings from a machine that says this.

2 The skeleton

2.1 Files

- `shaders/circle.frag`, `noise.glsl`, `sdf.frag`, `heightmap.vert`, `heightmap.frag`, `mytoy.frag`: the six files you edit, and the only ones. Five TASKs live in them.
- `shaders/fullscreen.vert`: three vertices out of `gl_VertexIndex`, no vertex input at all. Given, and used by Parts I, II, III and V.
- `shaders/toy.glsl`: the uniform block, below. `shadertoy.glsl`: Part V's Shader-Toy wrapper. `terrain.frag`: Part II's picture, which calls the noise you write. `heightmap_flat.vert` and `heightmap_flat.frag`: Part IV's comparison, the same noise moved to the fragment stage. Given; read them, they are short.
- `src/circle.cpp` and its five siblings: one program per part, about 150 lines each. Each renders, probes a few pixels, compares against the CPU, times the pass, then opens the window. When a check value here surprises you, read the part that printed it.
- `src/core/` and `src/utils/`: device, target, pipeline, shader compilation, the frame loop with hot reload, the GPU timer, the window, image comparison, and `Hash.h`, the C++ twin of the hash in `noise.glsl`. Complete; do not edit.
- `reference/`: four PNGs rendered at $t = 1$ from a correct solution. `report/`: where the S key and `make toy` save PNGs for your report. Created on first use, ignored by git.

2.2 The uniform block

One block, 176 bytes, the same for all five parts, declared once in `shaders/toy.glsl` and mirrored byte for byte by `src/utils/Toy.h`. You do not write it, but you read from it in every shader, and the offsets are the `std140` rules of Lab 06 TASK 2 applied to a bigger struct.

Member	Type	Offset	Size	What it is
resolution	vec2	0	8	the target, 800×600
time	float	8	4	seconds, or whatever <code>--time</code> says
mode	uint	12	4	0 final, 1 to 3 debug views
mouse	vec4	16	16	xy in pixels, <i>y</i> down ; z is 1 while the left button is held
octaves	uint	32	4	Parts II, IV, V
knob	uint	36	4	the grid N in Part IV
knobf	float	40	4	the height amplitude in Part IV
pad0	float	44	4	so the next <code>mat4</code> starts at a multiple of 16
view	mat4	48	64	Part IV camera
proj	mat4	112	64	Part IV projection, reversed-Z
block size			176	

`pad0` is not decoration. A `vec4` and every column of a `mat4` start on a 16-byte boundary in `std140`, and `knobf` ends at byte 44. Delete the pad and the two sides disagree about where `view` begins, so your camera reads four bytes of `knobf`. `Toy.h` has a `static_assert` on every offset in the table so that this cannot happen quietly.

3 The fragment shader as a function

A fragment shader is a function from one pixel to one color. It gets `gl_FragCoord`, the pixel center, and the uniform block. It cannot loop over the image, it does not know what its neighbors are doing, and it runs half a million times per frame. Everything in Parts I to III is that one function written better.

Two coordinates. `uv` is the pixel as a fraction of the target, 0 to 1 across and down: right for sampling, wrong for shapes, because the target is not square. Divide by the *height* in both components instead and you get a centered, aspect-corrected p :

$$p = \frac{2 \text{gl_FragCoord.xy} - R}{R_y}, \quad p_x \in \left[-\frac{4}{3}, \frac{4}{3}\right], \quad p_y \in [-1, 1] \quad (1)$$

That is the line at the top of nearly every shader on ShaderToy, and it is §7.6.1 of your chapter verbatim. Note where $p_y = -1$ lands.

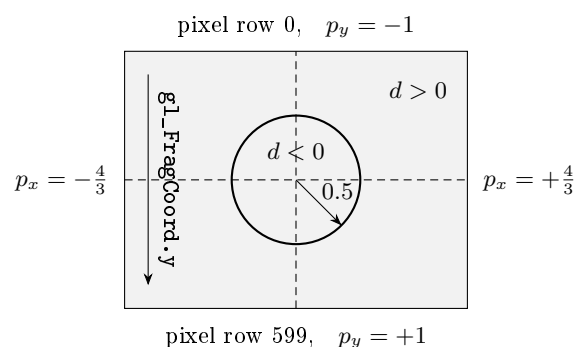


Figure 1: The p plane of Equation 1, drawn the way the screen is drawn. y counts *down* in Vulkan, so $p_y = -1$ is the top row and a shader ported from ShaderToy arrives upside down. The circle is the zero set of $d = \|p\| - 0.5$.

One number instead of a shape. A signed distance function returns the distance from p to the nearest surface, negative inside, zero on it, positive outside (§7.3.3, DEF-7.02). `length(p) - 0.5` is a disc; two of them combine with `min`. No mesh, no point-in-polygon test, and the same field gives you the outline, the fill, the glow and the antialiasing.

The edge. A shape cut with $d < 0.0$ has stair steps on its rim. `smoothstep` ramps across the rim instead (EQ-7.02):

$$\text{smoothstep}(e_0, e_1, x) = t^2(3 - 2t), \quad t = \text{clamp}\left(\frac{x - e_0}{e_1 - e_0}, 0, 1\right) \quad (2)$$

With $e_0 > e_1$ the ramp runs backwards, which is what you want when the inside of the shape is the *negative* side: `smoothstep(0.02, -0.02, d)` is 0 well outside and 1 well inside. But 0.02 is a distance in p units, and a p unit is 300 pixels here and 1080 on a 4K screen. Your antialiasing should be one pixel wide, not one number you picked.

fwidth, and why a lone pixel can hold a derivative. `fwidth(d)` is $|\partial d / \partial x| + |\partial d / \partial y|$: how much d changes between this pixel and the next. Substitute it for the constant and `smoothstep(w, -w, d)` is one pixel wide at every resolution.

How does a shader that only knows its own pixel produce a derivative? It does not: the GPU shades pixels in *quads*, blocks of 2×2 , and a derivative is the difference between neighbors in the same quad. Same hardware fact as Lab 06 Part III, where you measured a subgroup by making neighboring pixels branch differently. Two consequences: a quad straddling the edge of a triangle runs fragments that are thrown away afterwards, purely so their neighbors have something to differentiate against, and `fwidth` inside an `if` that not all four pixels take is undefined.

Debug by color. There is no `printf` in a fragment shader, so you output the number you want to see as a color (§7.1.5, F-7.01). Every shader here has a `u.mode` switch: 0 is the picture, 1 to 3 show a variable. Most of those views are given. You will find out what they are for the first time something is wrong.

Why the hash is integer arithmetic. Every noise tutorial, your chapter's §7.3.2 included, starts with `fract(sin(dot(p, ...)) * 43758.5453)`. It works, and it is not reproducible: `sin` of a large argument is where GPU vendors disagree most, so the same shader draws a different terrain on the machine next to you and no reference image can ever match. `noise.glsl` gives you a PCG integer hash instead. Integer multiply, shift and xor are exact and identical on every GPU and on the CPU, which is why the panel can compute the same number in C++ and hold your shader to it.

4 Part I: The pixel is a function

Run `make circle`. Black screen, and a panel saying **TASK 1 incomplete**. Four lines fix that, and there is no vertex buffer anywhere in this program.

4.1 TASK 1: circle.frag

Listing 1: `shaders/circle.frag`, the four TODOs

```

1 void main() {
2     // TODO (TASK 1a)
3     vec2 uv = vec2(0.0);
4
5     // TODO (TASK 1b)
```

```

6   vec2 p = vec2(0.0);
7
8   // TODO(TASK 1c)
9   float d = 1.0;
10
11  // TODO(TASK 1d)
12  vec3 col = vec3(0.0);
13
14  if      (u.mode == 1u) col = vec3(uv, 0.0);
15  else if (u.mode == 2u) col = vec3(d * 0.5 + 0.5);
16  else if (u.mode == 3u) col = vec3(fract(d * 10.0));
17
18  outColor = vec4(col, 1.0);
19 }

```

The four, in order: uv is the pixel as a fraction of the target, 0 to 1 across and down; p is Equation 1, centred and aspect corrected, so p_y runs -1 to 1 top to bottom, p_x runs $-\frac{4}{3}$ to $\frac{4}{3}$, and a circle stays round; d is the signed distance to a circle of radius 0.5 , negative inside, zero on the rim, positive outside; and col is $0.5 + 0.5 \cdot \cos(u.time + d \cdot 10.0 + \text{vec3}(0.0, 2.0, 4.0))$ cut to the disc by multiplying by $\text{smoothstep}(0.02, -0.02, d)$. Note the order of those two edges: the negative side is the inside. The three `u.mode` branches below are given, and are the debug views on keys 1, 2 and 3.

Check values. Work all four out before you run anything.

- **Coverage** = 15.9%. The disc has area $\pi \cdot 0.5^2 = 0.785$ and the visible plane is $2 \times \frac{8}{3} = 5.333$ square p units, so the disc is 14.7% of the image. The panel prints 15.9% because the `smoothstep` ramp starts at $d = 0.02$: a ring of circumference π and width 0.02 outside the rim is not quite black, and $\pi \cdot 0.02 / 5.333$ is the missing 1.2 points.
- **Center pixel** = (42, 77, 255). Pixel (400, 300) has `gl_FragCoord` = (400.5, 300.5), so $p = (\frac{1}{600}, \frac{1}{600})$ and $d = -0.4976$. At $t = 1$ the cosine's argument is $1 - 4.976 = -3.976$, and $0.5 + 0.5 \cos(-3.976 + (0, 2, 4)) = (0.164, 0.303, 1.000)$. Multiply by 255. The target is UNORM, so no sRGB curve sits between the shader and your color picker.
- **Mode 1 corners.** (0, 0) black, (799, 0) pure red, (0, 599) pure green. Decide *which corner of the screen* pixel (0, 599) is before you look. Your chapter's §7.1.5 puts the green corner at the top left. Here it is the bottom left, and that difference is write-up 1.1.
- **image MATCH**, against `reference/circle.png` at $t = 1$.

```

$ make circle
shaders      fullscreen.vert + circle.frag, no vertex buffer
uniform      176 bytes, time 1.000 s, mode 0
coverage     15.9% of the target, pi*0.25 / (2 * 8/3) = 14.7% before the AA ring
centre       pixel (400, 300) reads (42, 77, 255), cpu says (42, 77, 255)
mode 1       (0, 0) reads (0, 0, 0), expected (0, 0, 0)
mode 1       (799, 0) reads (255, 0, 0), expected (255, 0, 0)
mode 1       (0, 599) reads (0, 255, 0), expected (0, 255, 0)
image        MATCH (0.00% differ)
gpu time     0.032 ms median, 0.041 ms p95

```

4.2 Expected result

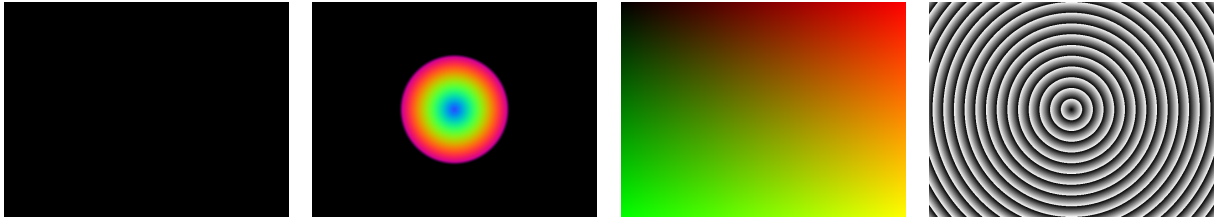


Figure 2: **Skeleton:** d never leaves 1.0, so every pixel is outside and the screen is black, coverage 0.0%, image DIFFERS (15.73% differ). **Mode 0:** expected, at $t = 1$. **Mode 1:** the uv debug view. Red grows to the right, green grows *downwards*. **Mode 3:** $\text{fract}(d \cdot 10)$, one contour ring every 0.1 units, evenly spaced and concentric because this field is the true distance. Keep that picture: in Part III it stops being true. Mode 2, not shown, is the raw field $d \cdot 0.5 + 0.5$.

Write-up 1.

1. Where the y flip that Lab 05's `viewportTransform()` used to do has gone, which corner of your mode 1 view is green, and the *one line* you would have to change in a shader copied from ShaderToy.
2. Why a fixed threshold, `smoothstep(0.01, -0.01, d)`, is resolution dependent while `smoothstep(w, -w, d)` is not. Give the width of each, in pixels, at 800×600 and at 3840×2160 .
3. What `fwidth(d)` is, and how the GPU can take a derivative of a value that each pixel computed for itself. Name the hardware fact, and the Lab 06 part where you measured it.

5 Part II: Noise, and what an octave costs

Run `make terrain`. The picture is given in `terrain.frag`, which colors a height in five bands. The height is `fbm`, and `fbm` is yours.

5.1 TASK 2: noise.glsl

This file is an *include*, not a shader. Parts II, IV and probably your Part V all pull it in with `#include <noise.glsl>`. Write it once, wrongly, and three parts are wrong together.

Listing 2: `shaders/noise.glsl`, given above the line, yours below

```
1  uint pcg(uint v) {
2      v = v * 747796405u + 2891336453u;
3      uint w = ((v >> ((v >> 28u) + 4u)) ^ v) * 277803737u;
4      return (w >> 22u) ^ w;
5  }
6
7  float hash(uvec2 p) { return float(pcg(p.x ^ pcg(p.y))) / 4294967296.0; }
8
9  // TODO(TASK 2a)
10 float value_noise(vec2 p) {
11     return 0.0;
12 }
13
14 // TODO(TASK 2b)
15 float fbm(vec2 p, uint octaves) {
16     return 0.0;
17 }
```

`pcg` and `hash` are given. `value_noise` hashes the four corners of the cell `p` falls in and blends between them: `floor` and `fract` give you the cell and the position inside it, the cell is cast with `uvec2(ivec2(i) + 1000)` so that a small negative cell does not wrap to the far end of the lattice, and the blend weight is the Hermite `f*f*(3.0 - 2.0*f)`, never `f` itself. `fbm` sums `octaves` calls to it, starting at amplitude 0.5 and frequency 1.0 and halving the amplitude while doubling the frequency each time:

$$\text{fbm}(p) = \sum_{i=0}^{n-1} A^{i+1} \cdot \text{noise}(p \cdot L^i), \quad A = 0.5, \quad L = 2.0 \quad (3)$$

That is EQ-7.03 with the constants this lab uses. Two properties of it are check values, so read it before you write it.

Check values.

- **The lattice probe reads 169, on every machine.** `terrain.frag` sets `p` to `(gl_FragCoord.xy - 0.5) / 100.0`, so pixel (300,200) is exactly $p = (3, 2)$, an integer lattice point. There the fractional part is zero, both blends collapse, and `value_noise` returns one hash and nothing else: `hash(uvec2(1003, 1002))`, offset by the +1000 above. The panel computes the same call in C++ and prints both. They agree to the last bit because the hash is integer arithmetic.
- **One octave is half.** `fbm(p, 1)` is $0.5 \times \text{value_noise}(p)$, so the same pixel reads 84. If it reads 169, your loop starts at amplitude 1.0.
- **The ceiling is 0.996.** The amplitudes are 0.5, 0.25, ..., and eight of them sum to $1 - 0.5^8 = 0.996$. No pixel can exceed it. The brightest pixel of the reference image is 0.914, comfortably under, because no point in the picture happens to sit on the peak of all eight octaves at once.
- **image MATCH at 6 octaves, against `reference/terrain.png`.**

```
$ make terrain
shaders      fullscreen.vert + terrain.frag, which includes noise.glsl
uniform      time 1.000 s, mode 0, octaves 6
lattice      pixel (300, 200) is p = (3, 2): value_noise reads 169,
              cpu hash(uvec2(1003, 1002)) is 169
one octave   fbm(p, 1) reads 84, half of 169 is 84
fbm ceiling  1 - 0.5^8 = 0.996, brightest pixel of the image 0.914
octaves 1    <your ms> ms median
...
octaves 8    <your ms> ms median
image        MATCH (0.00% differ) at 6 octaves
gpu time     0.288 ms median, 0.304 ms p95
```

Predict first. Before you run the sweep, write item 1 of Write-up 2: the ratio between 8 octaves and 1, and the shape of the curve. The answer is not interesting; being wrong in writing, and then saying why, is the exercise. The machine these notes were measured on gives roughly a straight line at about 0.026 ms per octave, but that is one GPU at one resolution, and the number you report must be the one your own panel printed.

5.2 Expected result

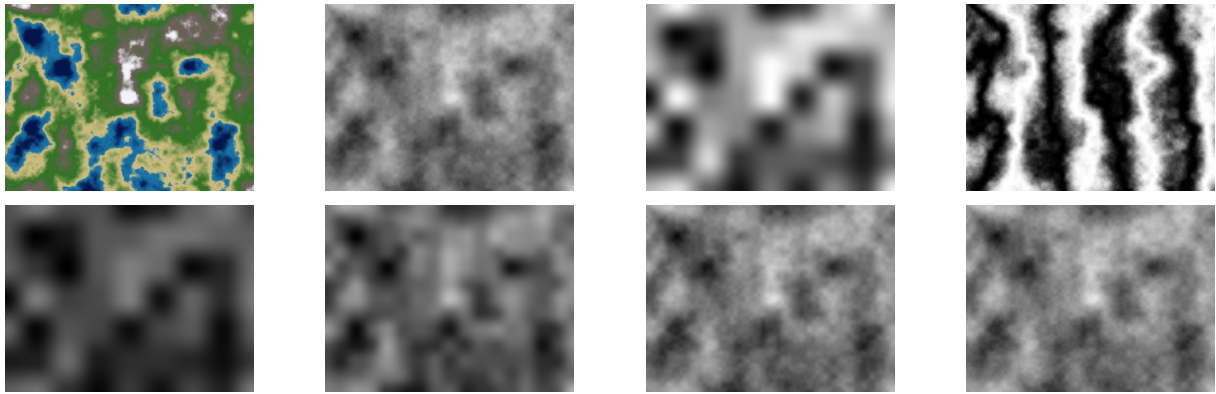


Figure 3: **Top row**, `terrain.frag` at 6 octaves: mode 0, the height bands drifting with time; mode 1, the same `fbm` as grey with no time, which is what the panel probes; mode 2, a single `value_noise` octave, one cell every 100 pixels; mode 3, marble. **Bottom row**, mode 1 at 1, 2, 4 and 8 octaves: each octave is half as tall and twice as fine, so the silhouette barely changes after the fourth while the cost keeps climbing. That gap is write-up 2.

Write-up 2.

1. *Before measuring*: your predicted ratio between 8 octaves and 1, the shape of the curve, and one line of reasoning.
2. Your eight measurements, the ms per octave, and whether the curve is linear. Compare with your prediction. If it is not linear, say what else the measurement is measuring.
3. The budget in §7.6's scenario is 0.2 ms. How many octaves can you afford at 800×600 , and how many at 4K, which is $17.3\times$ the pixels? State the assumption your extrapolation makes.
4. Why your friend's GPU draws a different terrain when the hash is the chapter's `fract(sin(dot(p, ...)) * 43758.5453)`, and why the integer hash does not have that problem.
5. Why Hermite $f^2(3 - 2f)$ and not plain f . Swap them, look at mode 2, and describe what appears.

6 Part III: Signed distance

Run `make sdf`. One fragment shader, no mesh, and a shape made of three primitives and two set operations. You write the primitives and the operation; the scene is dictated, because the panel knows the answer.

6.1 TASK 3: `sdf.frag`

Listing 3: `shaders/sdf.frag`, trimmed to the TODOs. `sdf_circle`, `op_union`, `op_subtract`, `op_intersect` and a `contours()` helper are given above this

```
1 // TODO(TASK 3a)
2 float sdf_box(vec2 p, vec2 hs) {
3     return 1.0;
4 }
5
```

```

6 // TODO(TASK 3b)
7 float sdf_rounded_box(vec2 p, vec2 hs, float r) {
8     return 1.0;
9 }
10
11 // TODO(TASK 3c)
12 float op_smooth_union(float d1, float d2, float k) {
13     return op_union(d1, d2);
14 }
15
16 // TODO(TASK 3d)
17 float scene(vec2 p) {
18     return 1.0;
19 }
20
21 void main() {
22     vec2 p = (2.0 * gl_FragCoord.xy - u.resolution) / u.resolution.y;
23     float d = scene(p);
24     float w = fwidth(d);
25
26     vec3 col = vec3(0.0);
27     if (u.mode == 0u) {
28         // TODO(TASK 3e)
29     }
30     ...
31 }

```

`sdf_box` is the exact distance to a box centred on the origin with half sizes `hs`: fold the plane into one quadrant with `abs`, then handle outside and inside separately. `sdf_rounded_box` is one subtraction away from it: shrink the half sizes by r first, then push the whole field out by r . `op_smooth_union` is the chapter's polynomial `smin`: blend the two distances with $h = \text{clamp}(0.5 + 0.5(d_2 - d_1)/k, 0, 1)$, take `mix(d2, d1, h)`, then pull the result in by $kh(1 - h)$. TASK 3e is `edge = smoothstep(w, -w, d)`, one pixel of antialiasing whatever the resolution, followed by `col = mix(vec3(0.0), vec3(0.9, 0.5, 0.2), edge)`: the background has to stay exactly black, because the panel measures coverage with it.

The scene, TASK 3d. It is dictated, because the panel knows the answer. Build exactly this, in this order:

- **body**: a rounded box at $(-0.35, 0)$, half size $(0.40, 0.25)$, corner radius 0.08.
- **ball**: a circle of radius 0.30 at `ball_center()`.
- **d**: body smooth-unioned with `ball`, $k = 0.20$.
- **hole**: a circle of radius 0.12 at $(-0.35, 0)$, subtracted from `d`. That subtraction is the return value.

`ball_center()` is given. It bobs on its own, and follows the cursor while the left button is down, using the same mapping as `main()`, so the ball lands under your mouse in every view. The panel and the reference use the bobbing position at $t = 1$, with no mouse.

Check values. The panel evaluates the same scene in C++ with GLM and compares, at four pixels chosen to be in four different places. Mode 2 writes $d \cdot 0.5 + 0.5$ into an 8-bit channel, so one level is $2/255 = 0.008$ of distance: the shader column is quantised and will not match the CPU column exactly. Anything further apart than 0.008 is a real disagreement.

Pixel	Where it is	shader d	cpu d	sign
(220,300)	inside the body	-0.129	-0.128	inside
(60,60)	outside everything	0.702	0.700	outside
(430,300)	in the blend	0.012	0.010	outside
(295,300)	inside the hole	0.114	0.118	outside

Two of those rows deserve a second look. The hole was *subtracted*, so a point inside it is outside the shape and d is positive: that is what $\max(d, -\text{hole})$ means. And the blend probe is the point of the part:

bound	in the blend the field says 0.010, the nearer primitive is 0.052 away: after a smooth union d is a bound, not a distance
-------	--

Nothing is within 0.010 of that pixel. The nearest surface, the corner of the rounded box, is 0.052 away. `op_smooth_union` subtracts $kh(1-h)$, which pulls the field down near the junction and invents a surface neither primitive has. The sign is still right everywhere and the value is still never larger than the true distance, which is all the rendering needs, but it has stopped being a distance. In a ray marcher that is the difference between stepping safely and stepping through the wall.

```
$ make sdf
shaders      fullscreen.vert + sdf.frag
uniform      time 1.000 s, mode 0
probe        (220, 300) inside the body      shader d -0.129   cpu d -0.128   inside
...
coverage     12.5% of the target is shape, the rest is exactly black
image        MATCH (0.00% differ)
gpu time     0.056 ms median, 0.057 ms p95
```

6.2 Expected result



Figure 4: **Skeleton:** `scene()` returns 1.0, the target is black. **Mode 0:** expected, coverage 12.5%. **Mode 1:** contours, one ring every 0.1; compare with mode 3 of Figure 2, where they were evenly spaced. Around the fillet they bunch up and bend, because the field there is no longer a distance. **Mode 2:** the raw field. **Mode 3:** `fwidth(d)` times 50, nearly flat, which is the antialiasing width TASK 3e needs; the bright seam is the crease where `max` changes its mind.

Write-up 3.

1. Why `min` is union and `max(d1, -d2)` is subtraction. One sentence each, in terms of what the distance to the combined shape must be.
2. What k does to the blend, and what it does to the contour lines in mode 1. Change it, screenshot two values, and say which one you would ship.
3. The field is the exact distance for a circle but only a bound after a smooth union. Quote your blend probe, say by how much it underestimates, and name one algorithm that would break because of it.

7 Part IV: The same noise, in the other stage

Run `make heightmap`. The host builds a flat $N \times N$ grid over $xz \in [-1, 1]$, indexed, with 32-bit indices, and gives you a camera in the uniform block. Your `fbm` now runs once per *vertex* and lifts the grid into terrain. This is Lab 06 Part II again, the same body in a different stage, except this time the body is yours.

7.1 TASK 4: heightmap.vert and heightmap.frag

Listing 4: `shaders/heightmap.vert` and `heightmap.frag`, trimmed to the TODOs

```
1 // TODO(TASK 4a)
2 float height(vec2 xz) {
3     return 0.0;
4 }
5
6 void main() {
7     vec3 pos = vec3(inXZ.x, height(inXZ), inXZ.y);
8
9     // TODO(TASK 4b)
10    vec3 n = vec3(0.0, 1.0, 0.0);
11
12    outWorld    = pos;
13    outNormal    = n;
14    outHeight    = clamp(pos.y / u.knobf, 0.0, 1.0);
15    gl_Position = u.proj * u.view * vec4(pos, 1.0);
16 }
17
18 // ---- heightmap.frag ----
19 // TODO(TASK 4c)
20 vec3 bands(float h) {
21     return vec3(0.5);
22 }
23
24 void main() {
25     vec3 n = normalize(inNormal);
26     vec3 col = vec3(0.5);
27     if (u.mode == 0u) {
28         // TODO(TASK 4c)
29     }
30     else if (u.mode == 1u) col = n * 0.5 + 0.5;
31     ...
32 }
```

`height()` is Part II's `fbm`, one call per vertex now instead of one per pixel: `return fbm(xz*1.5 + vec2(1.7, 9.2) + u.time*0.05, u.octaves)` scaled by `u.knobf`, the amplitude. TASK 4b takes the normal by central differences one cell apart, and one cell is `float e = 2.0 / float(u.knob)`:

```
n = normalize(vec3(height(xz - vec2(e, 0.0)) - height(xz + vec2(e, 0.0)),
                    2.0 * e,
                    height(xz - vec2(0.0, e)) - height(xz + vec2(0.0, e))));
```

That is two more `height()` calls, so three per vertex. Count them before you measure. In the fragment shader, `bands()` is the five-band ramp from `terrain.frag`, copied across unchanged, and mode 0 is Lambert: `L = normalize(vec3(0.5, 1.0, 0.3))` and `col = bands(inHeight) * (0.15 + 0.85 * max(dot(n, L), 0.0))`, where the 0.15 is the ambient that keeps the shadowed side off pure black.

Check values.

- **The mesh.** At $N = 256$: $N^2 = 65536$ vertices, $6(N - 1)^2 = 390150$ indices, 130050 triangles. Two triangles per cell, and every interior vertex shared by six of them: Lab 06 Part V at a scale where it matters.
- **The normal debug view.** Flat ground reads exactly (128, 255, 128), because $n = (0, 1, 0)$ maps to 0.5, 1.0, 0.5. The reference reads (149, 244, 81) at the center pixel. If yours reads (128, 255, 128) the terrain is lifted but TASK 4b is not done.
- **Invocation counts.** The vertex shader calls `height()` three times per vertex, once for the position and twice for the central differences: at $N = 64$ that is $3 \times 4096 = 12288$ evaluations of `fbm.heightmap_flat` draws the same grid flat and runs the same `fbm` once per covered fragment, 90772 of them, 7.4× more work from the same source. Write both counts down before you read the timings.
- **image MATCH** at $N = 256$, against `reference/heightmap.png`.

```
$ make heightmap
shaders      heightmap.vert + heightmap.frag, the vertex stage does fbm
grid         N 256: N*N = 65536 vertices, 6*(N-1)^2 = 390150 indices,
            130050 triangles
camera       eye (2.2, 1.6, 2.2), 45 deg, reversed-Z: clear 0,
            compare GREATER
normals      mode 1 centre pixel (149, 244, 81), flat ground would
            read (128, 255, 128)
coverage     18.1% of the target, 86734 fragments
vertex stage N 64   <your ms> ms median,      4096 vertices
vertex stage N 128  <your ms> ms median,     16384 vertices
vertex stage N 256  <your ms> ms median,     65536 vertices
vertex stage N 512  <your ms> ms median,    262144 vertices
fragment stage N 64  <your ms> ms median,      90772 fragments run the same fbm
image        MATCH (0.00% differ) at N 256
gpu time     0.325 ms median, 0.391 ms p95
```

Predict first. Item 1 of Write-up 4 asks you to rank all five rows, fastest to slowest, with one line of reasoning each *in invocation counts*, before you run it. The fragment row is the interesting one: it is the smallest grid in the table and it is not the cheapest.

7.2 Expected result

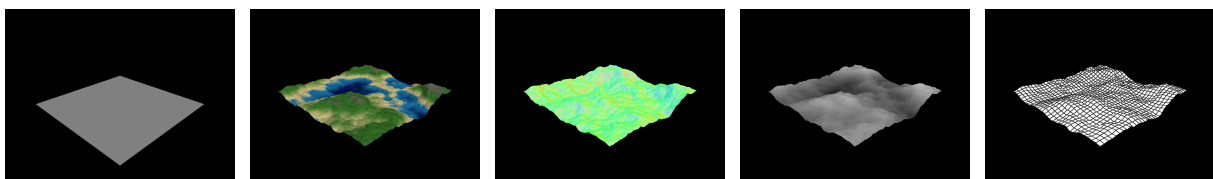


Figure 5: **Skeleton:** a flat grey plane, coverage 18.9%, normals exactly (128, 255, 128), image DIFFERS (23.41% differ). **Mode 0:** expected, at $N = 256$ and 6 octaves. **Mode 1:** the normals as color, green where the ground is level, red and blue on the slopes. **Mode 3:** height as grey, Part II's `fbm` seen from the side. **Mode 2,** on the right, draws one line per grid cell, here at $N = 32$, where you can count the cells and see a silhouette made of straight segments. At $N = 256$ the same view is nearly black: a cell is smaller than a pixel, so the lines fall between the samples. Press [and] to walk between the two.

Write-up 4.

1. *Before measuring:* rank the five rows fastest to slowest, with one line of reasoning each in invocation counts.

2. Your five rows, with the invocation count beside each, against your ranking. Explain the ratio between the $N = 64$ vertex row and the $N = 64$ fragment row using nothing but those counts and the three calls per vertex. Then explain why the vertex rows do not quite quadruple when N doubles.
3. Why the finite difference normal needs three `height()` calls, and what $dFdx/dFdy$ of the world position in the fragment shader would have given you instead. Say what the picture would look like.
4. What happens to the silhouette when N drops below the frequency of the noise. Press [until you can see it, screenshot it, and name what the same failure is called when it happens to a texture.

8 Part V: Your ShaderToy

Run `make toy`. It draws `mytoy.frag` through `shadertoy.glsl`, a wrapper that spells ShaderToy's names on top of our uniform block: `iResolution`, `iTime`, `iMouse`, and one extra, `iMode`, for your debug view. You write `mainImage`, the same entry point as §7.6.1.

Listing 5: `shaders/mytoy.frag`, the whole file

```

1 #version 450
2 #extension GL_GOOGLE_include_directive : require
3
4 #include <shadertoy.glsl>
5 #include <noise.glsl>
6
7 // TODO(TASK 5)
8 void mainImage(out vec4 fragColor, in vec2 fragCoord) {
9     vec2 uv = fragCoord / iResolution.xy;
10    vec3 col = 0.5 + 0.5 * cos(iTime + uv.yx + vec3(0.0, 2.0, 4.0));
11    fragColor = vec4(col, 1.0);
12 }
```

Do the copy-paste test first. Open <https://www.shadertoy.com>, pick any shader that uses no `iChannel` textures, copy its `mainImage` into `mytoy.frag`, and save with the window open. It should run, the right way up, first try. That is the point of the wrapper: it calls `mainImage(outColor, vec2(gl_FragCoord.x, u.resolution.y - gl_FragCoord.y))`, flipping y exactly once, so everything inside `mainImage` lives in ShaderToy's world where the origin is bottom left. If you flip y again yourself, you undo it. This is write-up 1.1 cashed in. Then delete it: the copy-paste test is a two minute check that your toolchain speaks the internet's dialect, not the deliverable.

TASK 5: the rules. An original animated shader, with five requirements. The panel checks four of them by reading your source, and the fifth with a clock.

1. It animates with `iTime` and reacts to `iMouse`. Drag in the window and something must change.
2. At least one SDF from Part III. `uses sdf_` is a grep for that prefix, so copy the functions you need into `mytoy.frag` and keep the names.
3. At least one `fbm` from Part II. The `noise.glsl` include is already in the file.
4. A debug view under `iMode`. Press 1, 2, 3 and see three different pictures.
5. Under 2.0 ms median at 800×600 , on your machine.

```

$ make toy
shaders      fullscreen.vert + mytoy.frag, through shadertoy.glsl
uniform      time 1.000 s, mode 0, octaves 5
gpu time     0.297 ms median, 0.405 ms p95      <- one shader, not yours
budget       2.000 ms at 800x600: PASS
uses noise.glsl  yes
uses sdf_        yes
uses iMouse      yes
uses iMode       yes
TODO removed    yes
coverage        100.0% of the target is not black
frames         report/toy_0.png, toy_1.png, toy_2.png

```

The three frames are written every run, at $t = 0, 1$ and 2 : your submission, in the style of F-7.02, the same shader at three times so a reader can see what moves.



Figure 6: One answer to TASK 5 at $t = 0, 1$ and 2 : a box smooth-unioned with four circles, three on Lissajous paths and one on the mouse, floating in an **fbm** whose sample position is itself displaced by an **fbm**. Every ingredient is from Parts II and III. Yours should not look like this one.

Budget and scope. 2.0 ms is generous and you can still blow it: two **fbm** calls at 8 octaves each is 16 noise evaluations per pixel. Measure, then cut, starting with octaves, because Figure 3 shows how little the last three buy you. Stop after 45 minutes in the lab and finish at home if you want to: this part is deliberately open, which makes it the easiest place in the course to lose an afternoon.

Visual taste is not graded: a shader that meets the five rules and is honestly explained is a complete answer.

Write-up 5.

1. Your three frames, and a paragraph on how the effect works *in maths*: one sentence per term of the picture, naming the variable in your code.
2. The cost breakdown. Comment out your most expensive term, report the median before and after, and give it as a fraction of the frame.
3. One thing you changed to fit the budget, and what it cost you visually.

9 Submission

Submit your **shaders/** directory and a report, in any format you like (Google Docs, Word, PDF), containing:

- **Six shader files:** **circle.frag**, **noise.glsl**, **sdf.frag**, **heightmap.vert**, **heightmap.frag** and **mytoy.frag**. Nothing else in the repository should have changed.
- **Your machine:** the **make info** screenshot, with every TASK line reading **done**. Every number you report is a statement about this machine, not about GPUs in general.

- **Screenshots:** the panel of each of the five parts, the four debug modes of at least one shader, and your Part V frames `report/toy_0.png`, `toy_1.png`, `toy_2.png`.
- **Five write-ups:** the numbered questions under each part. Answer from what is on your screen and the equations here, not from a general description of the topic. A number that contradicts the chapter, reported honestly and explained from your hardware, is a good answer; one that matches it with no evidence is not.
- **AI prompt log:** one entry per prompt, each with the **prompt**, the **model**, the **idea behind it** (what you needed and what you already knew) and the **verification** (how you checked it: the check values here, the panel's CPU column, the reference image, reading the code, working the coverage out on paper).

A log entry like “wrote the noise function for me” with empty idea and verification fields does not meet this requirement. Asking AI to write a shader is permitted; the write-ups and those two fields are where you show your own thinking. Anything you submit should survive a follow-up question in person, and in this lab it will: the last ten minutes of the session are for exactly that, and for Part V the question will be to change one constant in your shader and say what happens before you save.

10 References

- Akenine-Möller, T., Haines, E., Hoffman, N. (2018): *Real-Time Rendering*, 4th ed., Ch. 3 and Ch. 23. The programmable stages of Section 3, and the 2×2 quad behind `fwidth`.
- Quilez, I., 2D distance functions and fBM. The source of every primitive in Part III and of the polynomial smooth minimum, with the derivations the chapter leaves out.
- ShaderToy. Part V's dialect, and the place the copy-paste test comes from.
- Vulkan Specification, §Derivative Operations. Why `fwidth` is only defined when the whole quad agrees, and what *helper invocation* means.
- glslc and the glslang documentation. The `#include` extension this lab's `.glsl` files rely on, and what `-I` does to the search order.
- Vulkan Memory Layout appendix. The `std140` rules behind the 176-byte block in Section 2, and behind `pad0`.